

Nested iterator hint sheet

LeetCode 341 in Rust

Numbers are packed inside lists, and those lists can contain more lists. Your job is to deliver the numbers in their original left-to-right order. Choose one route, use its starter, and borrow every hint you need.

Choose how to unpack

Your choice	Route A Unpack first	Route B Open as needed
What you build	A flat vector of integers and a position.	A stack of integers and unopened lists.
When work happens	new walks through the entire input.	new reverses the outer list; has_next opens lists when needed.
What you gain	Simple next and has_next methods.	No complete flat result built upfront; can defer unused traversal.
What you manage	Recursion and a shared output vector.	Stack order and preserving the next integer.
Start here if	You want to follow the nested structure with recursion.	You want to reuse push and pop from the parentheses problem.

Both routes are valid. Choose A on page 2 or B on page 3. You may switch. Implement one; use page 4 to test it and explain your choice. Neither route is automatically faster or uses less memory in every case.

Meet the two kinds of item

```
pub enum NestedInteger {  
    Int(i32),  
    List(Vec<NestedInteger>),  
}  
use NestedInteger::*;
```

LeetCode already provides the enum. Keep its definition there; add only the use line and your chosen solution. `Int(number)` contains an `i32`; `List(inner)` contains a `Vec<NestedInteger>`. A list may be empty.

```
// The input [2, [5]] is represented as:  
vec![Int(2), List(vec![Int(5)])]
```

Unlike the parentheses task, the input already has enum values. There are no characters to convert. Every vector element has the same type, `NestedInteger`, even when the variants differ.

The promise both routes must keep

Method	Promise
<code>new(list)</code>	Remember enough state to deliver the integers in order.
<code>has_next()</code>	Say whether an integer remains. Checking twice must not skip it.
<code>next()</code>	Return one integer and advance. Call only when an integer remains.

Think of `has_next()` as asking whether another number is available. `next()` is the action that takes it. These are the problem methods; no Rust `Iterator` trait implementation is required.

Route A Unpack first

Open every list during construction and save the integers. Then keep a bookmark called `position`. For `[2, [5]]`, construction leaves `values = [2, 5]` and `position = 0`.

```
use NestedInteger::*;

struct NestedIterator {
    values: Vec<i32>,
    position: usize,
}

impl NestedIterator {
    fn new(nested_list: Vec<NestedInteger>) -> Self {
        let mut values = vec![];
        Self::flatten(nested_list, &mut values);
        Self { values, position: 0 }
    }

    fn flatten(list: Vec<NestedInteger>, values: &mut Vec<i32>) {
        for item in list {
            match item {
                Int(number) => todo!("A1"),
                List(inner) => todo!("A2"),
            }
        }
    }

    fn next(&mut self) -> i32 {
        let number = self.values[self.position];
        todo!("A3");
        number
    }

    fn has_next(&self) -> bool {
        todo!("A4")
    }
}
```

Rescue lines for the four gaps

Try describing the action first, then use its line. Replace only the `todo! ( . . . )` call; keep the surrounding commas or semicolons. An unfinished `todo! ( )` panics if reached.

Gap	Action and exact replacement
A1	Save the integer: <code>values.push(number)</code>
A2	Visit that inner list: <code>Self::flatten(inner, values)</code>
A3	Move the bookmark: <code>self.position += 1</code>
A4	Is the bookmark inside the vector? <code>self.position < self.values.len()</code>

`&mut values` lets the helper append to the same output vector. The recursive call receives `values`, which is already that mutable reference. An empty list runs the loop zero times.

Checkpoint: Why does `has_next ( )` use `<`, not `<=`? Why do we read the number before moving the bookmark? Then go to page 4.

Route B Open as needed

Keep pending work on a stack. Its right end is the top. To deliver 1, 2, 3, arrange the stack as [3, 2, 1]. Reverse each list when adding its children, so the leftmost child comes out first.

```
use NestedInteger::*;

struct NestedIterator {
    stack: Vec<NestedInteger>,
}

impl NestedIterator {
    fn new(mut nested_list: Vec<NestedInteger>) -> Self {
        todo!("B1");
        Self { stack: nested_list }
    }

    fn has_next(&mut self) -> bool {
        while let Some(item) = self.stack.pop() {
            match item {
                Int(number) => {
                    todo!("B2");
                    return true;
                }
                List(inner) => {
                    todo!("B3");
                }
            }
        }
        false
    }

    fn next(&mut self) -> i32 {
        self.has_next();
        match self.stack.pop().unwrap() {
            Int(number) => number,
            List(_) => unreachable!(),
        }
    }
}
```

Rescue lines for the three gaps

Gap	Action and exact replacement
B1	Put the first outer item on top: <code>nested_list.reverse()</code>
B2	Put the integer back before saying yes: <code>self.stack.push(Int(number))</code>
B3	Add children in reverse: <code>self.stack.extend(inner.into_iter().rev())</code>

Replace only the `todo!(...)` call. `pop()` yields `Some(item)` or `None`. `while let` keeps searching while an item exists. Finding an empty list adds nothing; the loop continues.

`has_next()` must restore an integer it finds. `next()` prepares the stack, then takes that integer. The `unwrap()` is valid under the promise that `next()` is called only when a number remains; see page 4 for the syntax and a loop alternative for B3.

Test your route and tell its story

Try these deliveries

Nested input	Integers delivered	What it checks
[[1,1],2,[1,1]]	[1,1,2,1,1]	Repeated values and sibling lists
[1,[4,[6]]]	[1,4,6]	Several nesting levels
[[],[[]],[[]]]	[]	Lists exist, but no integers exist
[[],1,[],[2,[]]]	[1,2]	Skip empty lists and keep looking
[[2,1],3]	[2,1,3]	Preserve order rather than sorting
[-1,[0,-1]]	[-1,0,-1]	Zero, negatives, and duplicates
[1,[[]]]	[1]	Only empty lists remain at the end

Use the nested inputs as LeetCode test cases. Also trace this call sequence on `[[], 1, [[]], 2]`:

```
has_next() // true
has_next() // true again; 1 must still be waiting
next()     // 1
has_next() // true
next()     // 2
has_next() // false
has_next() // false again
```

A small toolbox for either route

Rust spelling	What to read it as
<code>Self</code>	The type whose impl we are in: <code>NestedIterator</code> .
<code>&self</code> / <code>&mut self</code>	Read the object / allow changes to its state.
<code>Some(item)</code> / <code>None</code>	An item was returned / no item was returned.
<code>while let Some(item) = ...</code>	Repeat while that expression produces an item.
<code>inner.into_iter().rev()</code>	Move the children out of inner, in reverse order.
<code>unwrap()</code>	Extract the present value; panic if there is none.
<code>unreachable!()</code>	Panic if a branch our logic says cannot happen is reached.

Optional swap for B3

Prefer seeing each push? Replace the B3 statement with this loop. It does the same work as `extend`; use one form, not both.

```
for child in inner.into_iter().rev() {
    self.stack.push(child);
}
```

Finish with an explanation

Show that your chosen implementation passes the tests. Then explain: What does your struct remember? When does it open nested lists? What prevents repeated checks from losing a number? Name one benefit and one cost of your route.

If you finish early, trace the other route after taking only the first number. Which lists has it visited? You do not need to implement both routes.